ME560 Literature Review: Monte Carlo Tree Search Guided RL

Andrew Falcone

This literature review examines methods for solving turn-based games using reinforcement learning (RL). We begin with model-free approaches, focusing on temporal difference learning as demonstrated in *Temporal Difference Learning and TD-Gammon* [1]. We then consider Monte Carlo planning methods, specifically the bandit-based approach introduced by *Bandit Based Monte Carlo Planning* [2], which applies Upper Confidence Trees (UCT) to efficiently explore large search spaces and improve convergence relative to traditional α-β pruning methods used in minimax search. Finally, we analyze the general reinforcement learning framework presented in *Mastering the Game of Go without Human Knowledge* [3] and its extension to multiple domains in *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm* [4].

The key idea of *Temporal Difference Learning and TD-Gammon* [1] is to apply the temporal difference learning algorithm with a neural network value estimator so that the program can learn how to play Backgammon through self play, rather than relying on a database of expert moves. In this approach, the neural network approximates the value of the position, while temporal difference learning updates the networks's weights by assigning credit or blame for sequences of moves as games unfold. Importantly, decisions are made without a search or lookahead and moves are made from directly evaluating neural network value function. This is a difficult problem to solve for two main reasons. First, backgammon has a large state space of possible games including a stochastic element with dice being rolled each turn. The state space size, on the order of 10^20, makes using look-up tables or linear evaluation functions impractical as they are too limited to represent the game well. A second issue is that good play requires being able to learn proper credit placement. Distinguishing which moves were strong vs weak is critical for correctly modifying weights and value function estimation. TD-Gammon achieves near-expert performance using a relatively simple neural network with a single hidden layer, providing strong empirical evidence that temporal-difference learning with function approximation can solve complex decision-making problems.

In *Bandit Based Monte Carlo Planning* [2], the authors introduce Upper Confidence Trees (UTC), a method that significantly improves the efficiency of exploring large game state spaces. Unlike traditional minimax approaches with α–β pruning, which rely on heuristic evaluation functions and full tree traversal, UCT uses a Monte Carlo Tree Search (MCTS) approach that evaluates positions through repeated simulations. Monte Carlo methods estimate the value of a position by simulating games forward and averaging the outcomes, while MCTS organizes these simulations into a tree structure that is incrementally expanded over time. The key contribution of UCT is the use of a selection rule which balances exploration of less-visited actions with exploitation of actions that have already shown strong performance. This allows the algorithm to focus computational effort on the most promising parts of the search space. As a result, UCT can achieve stronger performance in large and complex domains without requiring carefully designed evaluation heuristics, while still converging toward optimal decisions with sufficient simulations.

Mastering the Game of Go without Human Knowledge [3] describes the evolution of DeepMind's AlphaGo models and the design decisions that enabled superhuman performance in a domain long considered one of the most challenging for artificial intelligence. Earlier versions, AlphaGo Fan and AlphaGo Lee, were initially trained using supervised learning to predict expert moves, and then refined through

policy-gradient reinforcement learning. These models were combined with Monte Carlo Tree Search (MCTS), which provided lookahead by exploring possible future game states and evaluating positions using both a policy network to guide move selection and a value network to estimate outcomes.

AlphaGo Zero differs from these earlier approaches in two key ways. First, it eliminates the use of supervised learning from human expert data, instead learning entirely through self-play. Second, it replaces the separate policy and value networks with a single neural network that outputs both move probabilities and value estimates. While AlphaGo Zero continues to use MCTS for lookahead, it removes the need for traditional Monte Carlo rollouts by relying entirely on the neural network's value predictions to evaluate positions, resulting in a more efficient and stable search process. This builds on UCT-based search by replacing random simulations with guidance from a learned model, allowing the algorithm to focus on promising moves and evaluate positions more efficiently.

Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm [4] extends the approach of *AlphaGo* by demonstrating that the same underlying algorithm can be generalized, achieving superhuman performance in several games without reliance on expert data. DeepMind's AlphaZero model was shown to surpass state-of-the-art systems, achieving higher Elo ratings than Stockfish in chess, Elmo in shogi, and even earlier versions of AlphaGo in Go. Similar to AlphaGo Zero, AlphaZero combines a deep neural network with Monte Carlo Tree Search (MCTS), using the network to guide exploration and evaluate positions during search. This highlights that learned representations, when combined with search-based planning, can produce strong general-purpose decision-making systems across a range of complex environments.

While the Alpha Zero framework is clearly described at a conceptual level, full replication of the reported results is not feasible in practice due to the significant computational resources required, and the lack of publicly available training pipelines at comparable scale. The work is best characterized as applied research, as it demonstrates a complete system achieving superhuman performance in simulated environments, though it also introduces a broadly applicable methodology combining learning and planning. All results are obtained through self-play in deterministic game environments, with validation based on performance against existing benchmark systems rather than physical experimentation, and without consideration of real-world noise typically encountered in control applications. This work builds upon foundational ideas in temporal-difference learning and Monte Carlo tree search, and is further extended in subsequent research demonstrating its generalization across domains, indicating a broader shift toward integrating learning-based representations with search-based planning in reinforcement learning.

[1] G. Tesauro, "Temporal Difference Learning and TD-Gammon," Communications of the ACM, vol. 38, no. 3, pp. 58–68, 1995.

[2] L. Kocsis and C. Szepesvári, "Bandit Based Monte-Carlo Planning," in European Conference on Machine Learning (ECML), 2006.

[3] D. Silver et al., "Mastering the Game of Go without Human Knowledge," Nature, vol. 550, pp. 354–359, 2017.

[4] D. Silver et al., "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," arXiv:1712.01815, 2018.